

Week3 KMP & 集合论与位运算优化

—— 一个人能走多远不取决于顺境中能走多快，取决于在逆境中能否找回曾经的自己。

Updated 2026-03-17 12:10 GMT+8

Compiled by Hongfei Yan (2025 Spring)

https://en.wikipedia.org/wiki/Knuth%E2%80%93Morris%E2%80%93Pratt_algorithm

在计算机科学中，**Knuth-Morris-Pratt 算法**（简称 **KMP 算法**）是一种字符串查找算法。该算法通过观察到这样一个关键点来查找主文本字符串 **s** 中是否存在一个“单词”或子字符串 **w**：**当发生字符不匹配时，单词 **w** 本身已经包含了足够的信息，可以确定下一次可能的匹配位置，从而跳过对先前已匹配字符的重复检查。**

字符串匹配是许多初学者接触算法时的第一道“逻辑门槛”。传统的暴力匹配法（Naive Algorithm）虽然直观，但在处理大规模文本时显得极其笨拙。

我们将一起拆解 **Knuth-Morris-Pratt (KMP)** 算法。它不仅是历史上第一个实现线性时间复杂度的字符串匹配算法，更体现了算法设计的核心智慧：**通过对模式串（Pattern）的自我深度剖析，实现主串指针的“永不回头”。**

1 KMP

1.1 算法背景：告别“暴力”与“重复”

在 KMP 诞生之前，程序员们习惯于用“暴力”解决问题。1970年，Knuth、Morris 与 Pratt 独立发现了这个线性算法，Matiyasevich 也曾从自动机理论角度得出类似结论。它的出现，标志着字符串处理从 $O(m \times n)$ 时代跨入了 $O(m + n)$ 时代。

暴力算法的核心痛点：指针 **i 的频繁回退（Backtracking）**

在暴力算法中，一旦在比对过程中发现不匹配，主串指针 **i** 必须被迫回退到本次匹配起始位置的下一个字符。这种“推倒重来”的行为在处理具有高度重复性的字符串（如基因序列）时，会产生海量的无效重复劳动。

- **暴力匹配效率：** $O(m \times n)$ (n : 文本长度, m : 模式串长度)。
- **KMP 效率：** $O(m + n)$, 实现了真正的线性扫描。

过渡： 为了消除这种无效的回退，KMP 引入了一个极其聪明的预处理机制——**LPS 表**。它就像一份“跳跃导航图”，告诉我们失败时该去向何方。

1.2 核心预处理：LPS（最长相等前后缀）表

KMP 的核心在于：在匹配之前，先让模式串进行“自省”。我们需要计算 **LPS（Longest Proper Prefix which is also Suffix）**。

关键定义

- 真前缀 (Proper Prefix): 包含首字符、但不包含字符串本身的子串。例如 "ABCD" 的真前缀有 "A", "AB", "ABC", 但不包括 "ABCD"。
- 真后缀 (Proper Suffix): 包含末字符、但不包含字符串本身的子串。
- LPS 长度: 一个子串中, 同时身为“真前缀”和“真后缀”的最长字符串的长度。

示例：模式串 "ABABC" 的 LPS 计算

通过对模式串的每一个子序列进行“对称性”观察：

子串 (Pattern[0...i])	真前缀 (Proper Prefix)	真后缀 (Proper Suffix)	LPS 长度
A	(空)	(空)	0
AB	A	B	0
ABA	A, AB	A, BA	1
ABAB	A, AB, ABA	B, AB, BAB	2
ABABC	A, AB, ABA, ABAB	C, BC, ABC, BABC	0

观察 AAAA 的 LPS 数组为 [0, 1, 2, 3]。这是因为 LPS 具有增量性质：当在 AAA (LPS=2) 后面再加一个 A 时，只要新字符能延续前缀的趋势，LPS 就会在之前的基础上加 1。

So What? LPS 表实际上是一个有限状态自动机。它告诉我们：如果在索引 j 处匹配失败，由于 j 之前的字符已经匹配成功，我们可以利用这些已知的“重复结构”，直接将模式串滑动到下一个可能的逻辑起点。

1.3 指针的华尔兹：双指针 (I & J) 协作流程

在 KMP 中，我们使用 0-based 索引 维护两个指针：i (主串指针) 和 j (模式串指针)。

移动规则

- 匹配成功时 (Text[i] == Pattern[j]): i 和 j 同时向右移动。
- 不匹配时 (Text[i] != Pattern[j]):
 - 主串指针 i: 永远不回退。
 - 模式串指针 j: 如果 j > 0, 则 j 调转到 lps[j-1] 的位置；如果 j == 0, 则 i 向前移动一步。

视觉模拟：逻辑跳跃的瞬间

假设在主串中匹配 ABABC，在索引 4 处发生失配：

Step 1: 发生失配 (Before Jump)

```
1 | Index:    0 1 2 3 4 5 6 7
2 | Text:     A B A B A B C A
3 | Pointer:           i (Text[4] = 'A')
4 | Pattern:  A B A B C
5 | Pointer:           j (Pattern[4] = 'C') <-- Mismatch!
```

Step 2: 执行跳跃 (After Jump) 根据 `lps[j-1]` 即 `lps[3]` (值为 2) , 将 `j` 重置为 2。这意味着模式串的前两个字符 `AB` 无需重扫, 直接对齐。

```
1 | Index:    0 1 2 3 4 5 6 7
2 | Text:     A B A B A B C A
3 | Pointer:           i (原地等待, 指向 'A')
4 | Pattern:    A B A B C
5 | Pointer:           j (指向 Pattern[2] = 'A') <-- 现在 Pattern[j] == Text[i]!
```

1.4 深度解析：破解“跳跃”逻辑

为什么跳到 `lps[j-1]`? 因为 `lps[j-1]` 的值代表了当前已匹配部分的对称重叠长度。跳跃后, 主串的后缀与模式串的前缀自动对齐, 从而避免了冗余检查。

对比展示：处理不匹配的行为

维度	暴力回溯 (Naive)	KMP 跳跃
主串指针 <code>i</code>	回退到上轮起点的下一位	绝不回退, 单向向前
模式串指针 <code>j</code>	重置为 0	重置为 <code>lps[j-1]</code>
最坏情况效率	$O(mn)$ (如 Text: <code>AAAAAB</code> , Pattern: <code>AAAB</code>)	$O(n)$ (同样的输入只需扫一遍)
核心逻辑	对结构“盲目”	深度利用模式串的周期对称性

```
1 | """
2 | compute_lps 函数用于计算模式字符串的LPS表。LPS表是一个数组，
3 | 其中的每个元素表示模式字符串中当前位置之前的子串的最长前缀后缀的长度。
4 | 该函数使用了两个指针 length 和 i，从模式字符串的第二个字符开始遍历。
5 | """
6 | def compute_lps(pattern):
7 |     """
8 |     计算pattern字符串的最长前缀后缀 (Longest Proper Prefix which is also Suffix)
    表
9 |     :param pattern: 模式字符串
10 |     :return: lps表
11 |     """
12 |
13 |     m = len(pattern)
```

```

14     lps = [0] * m # 初始化lps数组
15     length = 0 # 当前最长前后缀长度
16     for i in range(1, m): # 注意i从1开始, lps[0]永远是0
17         while length > 0 and pattern[i] != pattern[length]:
18             length = lps[length - 1] # 回退到上一个有效前后缀长度
19         if pattern[i] == pattern[length]:
20             length += 1
21         lps[i] = length
22
23     return lps
24
25 def kmp_search(text, pattern):
26     n = len(text)
27     m = len(pattern)
28     if m == 0:
29         return 0
30     lps = compute_lps(pattern)
31     matches = []
32
33     # 在 text 中查找 pattern
34     j = 0 # 模式串指针
35     for i in range(n): # 主串指针
36         while j > 0 and text[i] != pattern[j]:
37             j = lps[j - 1] # 模式串回退
38         if text[i] == pattern[j]:
39             j += 1
40         if j == m:
41             matches.append(i - j + 1) # 匹配成功
42             j = lps[j - 1] # 查找下一个匹配
43
44     return matches
45
46
47 text = "ABABABABCABABABABCABABABABC"
48 pattern = "ABABCABAB"
49 index = kmp_search(text, pattern)
50 print("pos matched: ", index)
51 # pos matched:  [4, 13]
52

```

1.5 高级洞察：周期性与循环节

KMP 的预处理不仅用于匹配，还能揭示字符串的几何构造。

引理：最小循环节 假设一个字符串的长度为 L (1-based 计数)，其对应的 LPS 数组最后一位为 $\text{lps}[L-1]$ 。若满足 $L \pmod{L - \text{lps}[L-1]} = 0$ ，则：

- 该字符串由一个长度为 $d = L - \text{lps}[L-1]$ 的子串重复构成。
- 该子串即为最小循环节，重复次数 $K = L / d$ 。

应用：这在数据压缩和周期检测中至关重要。例如 `ababab`，长度 6，`lps[5]=4`。6-4=2，且 $6 \pmod{2} = 0$ ，证明 `ab` 是其最小循环节。

1.5.1 最小循环元引理的证明

关于最小循环元引理（Minimum Cycle Element Lemma，在 KMP 算法中常用于解决周期性字符串问题），其证明逻辑建立在 KMP 算法的 π 表（或称 LPS 表、next 数组）的基础之上。

1. 前置定义

根据源文件， π 表（LPS 表）记录的是模式串中最长的真前缀同时也是真后缀的长度。

- 设字符串 S 的长度为 n 。
- 设 $next[n]$ （即 $\pi[n]$ ）为 S 的最长公共前后缀长度。
- 设 $L = n - next[n]$ 为潜在的循环节长度。

2. 最小循环元引理内容

根据我们的对话历史，引理指出：若 $n \pmod L = 0$ （即 n 能被 $n - next[n]$ 整除），则 L 是字符串 S 的最小循环元长度，且循环次数为 n/L 。

3. 证明过程

我们要证明：如果 n 可被 $n - k$ 整除（其中 $k = next[n]$ ），则长度为 $d = n - k$ 的前缀是 S 的循环元。

第一步：利用 LPS 的定义建立等价关系 根据 LPS 的定义， $S[1 \dots k] = S[n - k + 1 \dots n]$ 。这意味着：

- $S = S[n - k + 1]$
- $S = S[n - k + 2]$
- ...
- $S[k] = S[n]$ 由于 $d = n - k$ ，上述等式可以写作： $S[i] = S[i + d]$ ，对于所有 $1 \leq i \leq k$ 均成立。

第二步：推导周期性 因为 $S[i] = S[i + d]$ 对所有有效的 i 都成立，这实际上定义了一个步长为 d 的周期性规律。

- 字符串的第 1 个字符等于第 $1 + d$ 个字符，第 $1 + d$ 个字符等于第 $1 + 2d$ 个字符，以此类推。
- 如果 d 能整除 n ，那么整个字符串 S 就可以被划分为 n/d 个长度为 d 的块，且每一块都与前一块完全相同。

第三步：证明“最小性” 为什么 $d = n - next[n]$ 是最小的循环元？

- 在 KMP 算法中， $next[n]$ 存储的是最长的公共前后缀。
- 因为 $next[n]$ 是最大的，所以 $n - next[n]$ 必然是最小的。
- 如果存在更小的循环元长度 d' ，那么必然会存在一个更长的公共前后缀 $k' = n - d'$ ，这与 $next[n]$ 是“最长”的定义相矛盾。

4. 结论与应用

KMP 算法通过研究模式串自身的结构（即 π 表）来避免重复检查字符。利用这一引理，不仅能实现高效匹配，还能快速判断一个字符串是否由某个子串重复多次构成（如 `ababab` 的 $next = 4$ ，则 $6 - 4 = 2$ ，而 $6 \pmod 2 = 0$ ，结论是 `ab` 为循环元）。

用途：判断字符串是否由某个子串重复构成

基于这个引理，可以用来快速判断一个字符串是否可以由某个子串重复得到。例如：

```
1 def is_repeated_pattern(s: str) -> bool:
2     n = len(s)
3     next = [0] * (n + 1)
4     j = 0
5     for i in range(2, n + 1):
6         while j > 0 and s[j] != s[i-1]:
7             j = next[j]
8         if s[j] == s[i - 1]:
9             j += 1
10        next[i] = j
11
12    p = n - next[n]
13    return n % p == 0 and n != p
14
15 print(is_repeated_pattern("ababab")) # True
```

1.6 学习总结：KMP 的极简清单

要掌握 KMP，记住三大支柱：

1. **预处理（LPS）**：通过分析模式串的真前后缀，找出对称性，构建跳跃状态机。
2. **双指针不回退**：主串指针 `i` 是时间复杂度的守护者，它是单向流动的。
3. **利用已知信息**：匹配失败不是结束，而是利用已经匹配成功的“影子”，进行一次聪明的逻辑平移。

实际应用场景：KMP 广泛应用于 **DNA 序列分析**（寻找特定的基因序列）、**文本编辑器搜索**以及**代码查重系统**。

核心：模式串本身包含了足够的信息，来确定下一次匹配该从哪里开始。

1.7 编程题目

练习02406: 字符串乘方

KMP, <http://cs101.openjudge.cn/practice/02406/>

给定两个字符串 `a` 和 `b`, 我们定义 `a*b` 为他们的连接。例如，如果 `a="abc"` 而 `b="def"`，则 `a*b="abcdef"`。如果我们将连接考虑成乘法，一个非负整数的乘方将用一种通常的方式定义：`a0=""` (空字符串)，`a(n+1)=a*(an)`。

输入

每一个测试样例是一行可打印的字符作为输入，用s表示。s的长度至少为1，且不会超过一百万。最后的测试样例后面将是一个点号作为一行。

输出

对于每一个s，你应该打印最大的n，使得存在一个a，让 $s = a^n$

样例输入

```
1 | abcd
2 | aaaa
3 | ababab
4 | .
```

样例输出

```
1 | 1
2 | 4
3 | 3
```

提示: 本问题输入量很大，请用scanf代替cin，从而避免超时。

来源: Waterloo local 2002.07.01

```
1 | '''
2 | 使用KMP算法的部分知识，当字符串的长度能被提取的"base字符串"的长度整除时，
3 | 即可判断s可以被表示为a^n的形式，此时的n就是s的长度除以"base字符串"的长度。
4 | '''
5 |
6 | import sys
7 |
8 | while True:
9 |     s = sys.stdin.readline().strip()
10 |     if s == '.':
11 |         break
12 |     n = len(s)
13 |     next = [0] * len(s)
14 |     j = 0
15 |     for i in range(1, n):
16 |         while j > 0 and s[i] != s[j]:
17 |             j = next[j - 1]
18 |         if s[i] == s[j]:
19 |             j += 1
20 |         next[i] = j
21 |     p = len(s) - next[-1]
22 |     if n % p == 0:
23 |         print(n // p)
24 |     else:
25 |         print(1)
```

练习01961: 前缀中的周期

KMP, <http://cs101.openjudge.cn/practice/01961/>

<http://poj.org/problem?id=1961>

For each prefix of a given string S with N characters (each character has an ASCII code between 97 and 126, inclusive), we want to know whether the prefix is a periodic string. That is, for each i ($2 \leq i \leq N$) we want to know the largest $K > 1$ (if there is one) such that the prefix of S with length i can be written as A^K , that is A concatenated K times, for some string A . Of course, we also want to know the period K .

一个字符串的前缀是从第一个字符开始的连续若干个字符，例如"abaab"共有5个前缀，分别是a, ab, aba, abaa, abaab。

我们希望知道一个 N 位字符串 S 的前缀是否具有循环节。换言之，对于每一个从头开始的长度为 i （ i 大于1）的前缀，是否由重复出现的子串 A 组成，即 $AAA...A$ （ A 重复出现 K 次， K 大于1）。如果存在，请找出最短的循环节对应的 K 值（也就是这个前缀串的所有可能重复节中，最大的 K 值）。

输入

输入包括多组测试数据。每组测试数据包括两行。

第一行包括字符串 S 的长度 N （ $2 \leq N \leq 1\,000\,000$ ）。

第二行包括字符串 S 。

输入数据以只包括一个0的行作为结尾。

输出

对于每组测试数据，第一行输出 "Test case #" 和测试数据的编号。

接下来的每一行，输出前缀长度 i 和重复次数 K ，中间用一个空格隔开。前缀长度需要升序排列。

在每组测试数据的最后输出一个空行。

样例输入

```
1 3
2 aaa
3 12
4 aabaabaabaab
5 0
```

样例输出

```
1 Test case #1
2 2 2
3 3 3
4
5 Test case #2
6 2 2
7 6 2
8 9 3
9 12 4
```


【POJ1961】period, <https://www.cnblogs.com/ve-2021/p/9744139.html>

如果一个字符串S是由一个字符串T重复K次构成的，则称T是S的循环元。使K出现最大的字符串T称为S的最小循环元，此时的K称为最大循环次数。

现在给定一个长度为N的字符串S，对S的每一个前缀S[1~i]，如果它的最大循环次数大于1，则输出该循环的最小循环元长度和最大循环次数。

题解思路：

- 1) 与自己的前缀进行匹配，与KMP中的next数组的定义相同。next数组的定义是：字符串中以i结尾的子串与该字符串的前缀能匹配的最大长度。
- 2) 将字符串S与自身进行匹配，对于每个前缀，能匹配的条件即是：S[i-next[i]+1 ~ i]与S[1~next[i]]是相等的，并且不存在更大的next满足条件。
- 3) 当i-next[i]能整除i时，S[1 ~ i-next[i]]就是S[1 ~ i]的最小循环元。它的最大循环次数就是i/(i - next[i])。

这是刘汝佳《算法竞赛入门经典训练指南》上的原题（p213），用KMP构造状态转移表。在3.3.2 KMP算法。

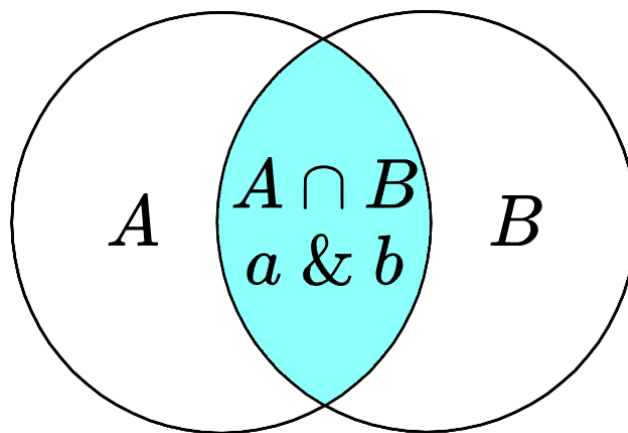
```
1  '''
2  gpt
3  这是一个字符串匹配问题，通常使用KMP算法（Knuth-Morris-Pratt算法）来解决。
4  使用了 Knuth-Morris-Pratt 算法来寻找字符串的所有前缀，并检查它们是否由重复的子串组成，
5  如果是的话，就打印出前缀的长度和最大重复次数。
6  '''
7
8  # 得到字符串s的前缀值列表
9  def kmp_next(s):
10     # kmp算法计算最长相等前后缀
11     next = [0] * len(s)
12     j = 0
13     for i in range(1, len(s)):
14         while s[i] != s[j] and j > 0:
15             j = next[j - 1]
16         if s[i] == s[j]:
17             j += 1
18         next[i] = j
19     return next
20
21
22 def main():
23     case = 0
24     while True:
25         n = int(input().strip())
26         if n == 0:
27             break
28         s = input().strip()
29         case += 1
30         print("Test case #{0}".format(case))
31         next = kmp_next(s)
32         for i in range(2, len(s) + 1):
```

```

33         k = i - next[i - 1]    # 可能的重复子串的长度
34         if (i % k == 0) and i // k > 1:
35             print(i, i // k)
36     print()
37
38
39 if __name__ == "__main__":
40     main()
41

```

2 集合论与位运算优化



2.0 引言：从数学概念到高效计算

在初等数学中，我们学习了**集合论 (Set Theory)**。例如，集合 $S = \{0, 2, 3\}$ 。在编程实现中，通常使用哈希表 (`unordered_set` 或 `set`) 来存储集合元素。

然而，当集合中的元素范围较小且固定（如 0-63 之间的整数）时，我们可以利用**二进制位**在逻辑电路与数学运算之间的同构性，建立一座高效的“桥梁”。

从集合论到位运算，常见位运算技巧分类总结！

灵茶山艾府，<https://leetcode.cn/discuss/post/3571304/cong-ji-he-lun-dao-wei-yun-suan-cha-ng-ji-enve/>

2.1 集合与位运算的映射关系

我们可以将集合中的元素 i 映射到二进制数的第 i 位（从低到高）。

形式化定义： 包含非负整数的有限集合 S ，可压缩为一个非负整数 $f(S)$ ：

$$f(S) = \sum_{i \in S} 2^i$$

示例：

- 集合 $\{0, 2, 3\} \rightarrow$ 二进制 $1101_{(2)} \rightarrow$ 十进制 13。
- 操作优势：位运算是 CPU 的基础指令，其执行效率远高于哈希表的函数调用。

常用集合操作对照表

术语	集合符号	位运算 (C++/Python)	集合示例	位运算示例 (1101 与 0111)
交集	$A \cap B$	<code>a & b</code>	$\{0, 2, 3\} \cap \{0, 1, 2\} = \{0, 2\}$	<code>1101 & 0111 = 0101</code>
并集	$A \cup B$	<code>a b</code>	$\{0, 2, 3\} \cup \{0, 1, 2\} = \{0, 1, 2, 3\}$	<code>1101 0111 = 1111</code>
差集	$A \setminus B$	<code>a & (~b)</code>	$\{0, 2, 3\} \setminus \{0, 1, 2\} = \{3\}$	<code>1101 & (~0111) = 1000</code>
对称差	$A \Delta B$	<code>a ^ b</code>	仅属于 A 或 B 的元素	<code>1101 ^ 0111 = 1010</code>
包含于	$A \subseteq B$	<code>(a & b) == a</code>	检查 A 是否为 B 的子集	<code>(0101 & 0111) == 0101 (True)</code>

2.2 实战演练：倒排索引查询

倒排索引 (Inverted Index) 是搜索引擎的核心。给定一组词及其出现的文档列表，根据查询指令（包含、排除、无所谓）计算最终文档集。

- **1 (AND):** 文档必须包含该词。
- **-1 (NOT):** 文档必须排除该词。
- **0 (IGNORE):** 无所谓。

算法策略：集合运算优化

由于文档 ID 范围较大（32位整数），无法直接使用单整数位掩码。但其底层的逻辑与上述位运算完全一致。我们利用 Python 内置的高效 `set`（基于哈希优化）来模拟位运算逻辑。

关键逻辑：

1. 先求交：找到所有标记为 `1` 的集合，求其交集 A 。
2. 后求差：找到所有标记为 `-1` 的集合，求其并集 B 。
3. 最终结果： $Result = A \setminus B$ 。

练习M04093: 倒排索引查询

Inverted Index, <http://cs101.openjudge.cn/practice/04093/>

现在已经对一些文档求出了倒排索引，对于一些词得出了这些词在哪些文档中出现的列表。

要求对于倒排索引实现一些简单的查询，即查询某些词同时出现，或者有些词出现有些词不出现的文档有哪些。

输入

第一行包含一个数N， $1 \leq N \leq 100$ ，表示倒排索引表的数目。

接下来N行，每行第一个数 c_i ，表示这个词出现在了多少个文档中。接下来跟着 c_i 个数，表示出现在的文档编号，编号不一定有序。 $1 \leq c_i \leq 1000$ ，文档编号为32位整数。

接下来一行包含一个数M， $1 \leq M \leq 100$ ，表示查询的数目。

接下来M行每行N个数，每个数表示这个词要不要出现，1表示出现，-1表示不出现，0表示无所谓。数据保证每行至少出现一个1。

输出

共M行，每行对应一个查询。输出查询到的文档编号，按照编号升序输出。
如果查不到任何文档，输出"NOT FOUND"。

样例输入

```
1 3
2 3 1 2 3
3 1 2
4 1 3
5 3
6 1 1 1
7 1 -1 0
8 1 -1 -1
```

样例输出

```
1 NOT FOUND
2 1 3
3 1
```

在实际搜索引擎在处理基于倒排索引的查询时，搜索引擎确实会优先关注各个查询词的倒排表的合并和交集处理，而不是直接准备未出现文档的集合。这种方法更有效，特别是在处理大规模数据集时，因为它允许系统动态地调整和优化查询过程，特别是在有复杂查询逻辑（如多个词的组合、词的排除等）时。详细解释一下搜索引擎如何使用倒排索引来处理查询：

倒排索引查询的核心概念

1. 倒排索引结构：

- 对于每个词（token），都有一个关联的文档列表，这个列表通常是按文档编号排序的。
- 每个文档在列表中可能还会有附加信息，如词频、位置信息等。

2. 处理查询：

- 单词查询：对于单个词的查询，搜索引擎直接返回该词的倒排列表。
- 多词交集查询：对于包含多个词的查询，搜索引擎找到每个词的倒排列表，然后计算这些列表的交集。
这个交集代表了所有查询词都出现的文档集合。
- 复杂逻辑处理：对于包含逻辑运算（AND, OR, NOT）的查询，搜索引擎会结合使用集合的交集（AND）、并集（OR）和差集（NOT）操作来处理查询。特别是在处理 NOT 逻辑时，它并不是去查找那些未出现词的文档集合，而是从已经确定的结果集中排除含有这个词的文档。

更贴近实际搜索引擎的处理实现，如下：

```
1 import sys
2 input = sys.stdin.read
```

```

3 data = input().split()
4
5 index = 0
6 N = int(data[index])
7 index += 1
8
9 word_documents = []
10
11 # 读取每个词的倒排索引
12 for _ in range(N):
13     ci = int(data[index])
14     index += 1
15     documents = sorted(map(int, data[index:index + ci]))
16     index += ci
17     word_documents.append(documents)
18
19 M = int(data[index])
20 index += 1
21
22 results = []
23
24 # 处理每个查询
25 for _ in range(M):
26     query = list(map(int, data[index:index + N]))
27     index += N
28
29     # 集合存储各词的文档集合（使用交集获取所有词都出现的文档）
30     included_docs = []
31     excluded_docs = set()
32
33     # 解析查询条件
34     for i in range(N):
35         if query[i] == 1:
36             included_docs.append(word_documents[i])
37         elif query[i] == -1:
38             excluded_docs.update(word_documents[i])
39
40     # 仅在包含词时计算交集
41     if included_docs:
42         result_set = set(included_docs[0])
43         for docs in included_docs[1:]:
44             result_set.intersection_update(docs)
45         result_set.difference_update(excluded_docs)
46         final_docs = sorted(result_set)
47         results.append(" ".join(map(str, final_docs)) if final_docs
48 else "NOT FOUND")
49     else:
50         results.append("NOT FOUND")
51
52 # 输出所有查询结果
53 for result in results:
54     print(result)

```

利用集合运算来处理倒排索引查询。思路如下：

1. 建立倒排索引

对于每个词，读入出现该词的文档编号，将其存入一个集合中。

2. 处理查询

对于每个查询，按照查询向量（每个位置取值 1、-1、0）：

- 对于标记为 1 的词（必须出现），求这些词对应集合的交集。
- 对于标记为 -1 的词（必须不出现），从交集中减去这些词出现的文档编号。
- 0 的词不作限制。

3. 输出结果

将最终得到的候选文档集合按升序输出；如果为空，则输出 "NOT FOUND"。

```
1 n = int(input())
2 files = [] # 储存所有单词的文档归属数据
3 for i in range(n):
4     data = list(map(int, input().split()))
5     files.append(set(data[1:])) # 将每个单词对应的文档数据转化为集合，方便后续的操作
6
7 m = int(input())
8 for i in range(m):
9
10    data = list(map(int, input().split()))
11    t = data.index(1) # 第一个1要特殊处理，作为初始值
12    a = set(files[t]) # 所有有可能合题的文档（同样要储存在集合内）
13    for j in range(t + 1, n):
14        if data[j] == 1:
15            a = a & files[j] # 取交集（集合之间的n、-、u运算时间复杂度极低！）
16
17    b = set() # 所有一定不合题的文档（同样要储存在集合内）
18    for j in range(n):
19        if data[j] == -1:
20            b = b | files[j] # 取并集
21
22    # 注意：一定要先把所有要包含的文档全部放入a，所有不能包含的文档统一放入b，最后再a-b！
23    # 否则会损失一些b中的信息，例如如果是“无所谓、不能包含2、要包含12”，
24    # 如果一步一步使用交集和差集的话，则1和2都合题，这与第二个条件矛盾！因此不能一步一步来
25    ~
26    c = a - b # a与b的差集
27    finale = list(c)
28    if finale:
29        finale.sort()
30        print(" ".join(map(str, finale)))
31    else: print("NOT FOUND")
```

附录：

A.数字华容道原理分析

M30178:数字华容道 (Easy Version) - 原理

merge sort, binary indexed tree, <http://cs101.openjudge.cn/practice/30178/>

我们把数字华容道中最核心、也最抽象的数学原理，一步一步来拆解。

这是一个非常经典的数字华容道 (N-Puzzle 或 15-Puzzle 的推广版) 是否可解的问题。判断 $n \times n$ 数字华容道是否可解，可以通过计算盘面数字 (忽略空格 0) 的逆序对数来决定。

为了避免直接计算逆序对带来的 $O(N \log N)$ 的时间复杂度 (当 $n = 1000$ 时, $n^2 = 10^6$, 稍不留神可能会超时), 我们可以利用排列的一个重要代数性质:

一个排列逆序对的奇偶性, 等于将其拆解为若干个独立环 (Cycle) 后, $(N - \text{环的数量})$ 的奇偶性。这使得我们能在 $O(N)$ 的线形时间内极快地求出逆序对的奇偶性。

对于 $n \times n$ 的盘面, 设空格 0 所在的行为 r (下标从 0 开始计算), 我们求出除 0 以外数字序列逆序对的奇偶性 `inv_parity`:

- 当 n 为奇数时: 每次上下移动空格时, 其他数字的平移相当于跨过了偶数个数字, 逆序对奇偶性不会改变。因此, 起始状态必须与目标状态 (逆序对为偶数 0) 的奇偶性相同。
 - 可解条件: `inv_parity == 0` (逆序对为偶数)。
- 当 n 为偶数时: 每次上下移动空格时, 其他数字跨过了奇数个数字, 导致逆序对奇偶性改变, 同时空格所在行数 r 的奇偶性也改变。二者的奇偶性相互制约。目标状态是空格在最后一行 ($r = n - 1$, 即奇数行), 逆序对为 0 (偶数)。
 - 可解条件: `(inv_parity + r) % 2 == 1`。

这段逻辑是数字华容道中最核心、也最抽象的数学原理。把这个过程“可视化”, 一步一步来拆解。

1. 前置概念: 把盘面“拉平”

首先, 我们把盘面上的数字从左到右、从上到下排成一排, 并且在这个序列中去掉空格 0。

比如一个 3×3 的盘面:

1	1	2	3
2	4	0	5
3	6	7	8

拉平后, 去掉 0, 数字序列是: 1, 2, 3, 4, 5, 6, 7, 8。

这个序列的逆序对数就是用来判断是否有解的钥匙。

2. 移动空格 0 会发生什么?

我们在盘面上移动空格 0, 分为两种情况: 左右移动 和 上下移动。

- 左右移动 0:
比如 0 和 4 交换位置, 盘面变了, 但是在“拉平”后的数字序列中, 4 和其他数字的前后相对顺序完全没有改变。
👉 结论: 左右移动, 序列的逆序对数量完全不变。
- 上下移动 0:
假设 0 向上移动, 和 2 交换了位置。

交换前的盘面：

1	1	2	3
2	4	0	5
3	6	7	8

交换后的盘面（0和2换）：

1	1	0	3
2	4	2	5
3	6	7	8

交换前拉平的序列：1, 2, 3, 4, 5, 6, 7, 8

交换后拉平的序列：1, 3, 4, 2, 5, 6, 7, 8

仔细观察 2 的位置：它从 3 和 4 的前面，跑到了它们后面！

也就是说，在 $n \times n$ 的盘面中，上下移动 0，相当于让某个数字在拉平的序列中跨越了 $n - 1$ 个数字（在这个 3×3 例子中，跨越了 $3 - 1 = 2$ 个数字）。

3. 为什么 n 是奇数和偶数时，情况完全不同？

数学上有一个定理：一个数字在序列中每跨越 1 个数字，逆序对的数量就会增加 1 或减少 1（也就是奇偶性改变 1 次）。

- 当 n 是奇数时（比如 3×3 、 5×5 ）：

跨越的数字个数是 $n - 1$ ，这是一个偶数（比如跨越 2 个、4 个数字）。

跨越偶数个数字，逆序对的奇偶性会改变偶数次（比如变了 2 次等于没变）。

👉 结论：对于奇数盘面，无论怎么移动，逆序对的奇偶性永远不变（守恒）。

目标盘面的逆序对是 0（偶数），所以只要初始盘面的逆序对也是偶数（`inv_parity == 0`），就有解。

- 当 n 是偶数时（比如 4×4 、 6×6 ）：

跨越的数字个数是 $n - 1$ ，这是一个奇数（比如跨越 3 个、5 个数字）。

跨越奇数个数字，逆序对的奇偶性就会发生改变（偶数变奇数，奇数变偶数）。

👉 结论：对于偶数盘面，逆序对的奇偶性是不守恒的！动一下上下，奇偶性就反转一次。

4. n 为偶数时的“终极破局点”：行号 r

既然偶数盘面的逆序对奇偶性在变，我们怎么判断它有没有解呢？

聪明的数学家发现了一个同时在变的规律：

每当你上下移动一次 0：

- 逆序对的奇偶性 `inv_parity` 反转了一次。
- 空格 0 所在的行号 `r`，也刚好加了 1 或减了 1（意味着行号的奇偶性也反转了一次）。

如果两个东西每次都同时反转，那么它们的和的奇偶性就永远保持不变！

就像两个硬币同时翻面，正面的总数如果是奇数，翻完之后还是奇数。

这就意味着 `(inv_parity + r) % 2` 是一个永远不会变的固定值（守恒量）。

5. 最终公式是怎么来的？

既然 `(inv_parity + r) % 2` 在玩家怎么玩都不会变，那它在初始状态和最终目标状态，计算出来的结果必须一模一样！

我们来看最终目标状态（以 4×4 为例）：

1	1	2	3	4
2	5	6	7	8
3	9	10	11	12
4	13	14	15	0

1. 它的逆序对数量是 0，所以 $\text{inv_parity} = 0$ 。
2. 0 在最后一行，行号从 0 开始数的话，它是第 3 行（第一行是 0，第二行是 1，第三行是 2，第四行是 3），所以 $r = 3$ （或者推广到任意偶数盘面，最后一行总是 $n - 1$ ，即奇数）。

计算一下目标状态的守恒量：

$$(\text{inv_parity} + r) \% 2 = (0 + 3) \% 2 = 3 \% 2 = 1$$

结论大揭秘：

因为目标状态算出来的这个值是 1，而且玩家无论怎么平移方块，这个值都不会变。

所以，只有当你拿到的初始混乱盘面，算出来 $(\text{inv_parity} + r) \% 2$ 也等于 1 时，它才有可能被还原成目标状态。如果算出来是 0，无论你怎么努力，都永远拼不回去！

练习M30178: 数字华容道（Easy Version）

merge sort, binary indexed tree, <http://cs101.openjudge.cn/practice/30178/>

总时间限制：2000ms，单个样例点时间限制：1000ms，内存限制：262144kB

数字华容道的游戏玩法：在一个 $n \times n$ 的方格中放置 1 到 $n^2 - 1$ 的所有数字（每个数字占据一空间），剩下一个空格（输入中用 0 表示）。每次可以将与空格相邻的几个数字平移到空格处。游戏目标是尽可能快的将给定盘面还原到初始状态。（初始状态就是 1 到 $n^2 - 1$ 按顺序填充每一行，空格在右下角的盘面）

例如一个可能的初始盘面是：

1	1	2	3	4
2	5	6	7	8
3	9	10	11	12
4	15	13	14	0

你的目标是通过平移数字，将盘面变成

1	1	2	3	4
2	5	6	7	8
3	9	10	11	12
4	13	14	15	0

的样子。

一天，小K正准备玩数字华容道，但是他不小心把盘面中的数字小方格都打翻在地了。虽然他将所有的数字都乱序排入了盘面中，但是这样的盘面是不一定有解的，所以他想请你判断一下这个盘面是否有解。

输入

第 1 行：一个整数 n ，($2 \leq n \leq 1000$)。（你就不要问小K是怎么把一百万个小方格捡起来的了）

第 2 行到第 $n + 1$ 行： n 个整数，表示盘面。

输出

如果盘面有解，输出yes，否则输出no

样例输入

```
1 | 4
2 | 1 2 3 4
3 | 5 6 7 8
4 | 9 10 11 12
5 | 13 14 15 0
```

样例输出

```
1 | yes
```

（注：此题与Hard Version唯一的不同之处在于Easy Version的每一个输入文件都只有一个样例，而Hard Version有多个。）

这是一个经典的数字华容道（N-Puzzle）的可解性判定问题。

核心思路

判断盘面是否有解，依赖于**逆序对（Inversions）**的数量和空格（0）的位置。

1. 将盘面展开为一维数组：忽略空格（0），将剩下的 $N^2 - 1$ 个数字按顺序排成一列。
2. 计算逆序对（ I ）：在展开的数组中，如果前面的数字大于后面的数字，则称为一个逆序对。计算总数 I 。
3. 判定规则：
 - 情况 1： N 是奇数
 - 如果逆序对数量 I 是偶数，则有解。
 - 否则无解。
 - 情况 2： N 是偶数
 - 计算空格（0）所在的行号（从上往下数，0-indexed），设为 r 。
 - 计算空格距离底部的行数： $dist = (N - 1) - r$ 。
 - 如果 $(I + dist)$ 是偶数，则有解。
 - 否则无解。

算法复杂度

由于 N 最大为 1000，直接双重循环计算逆序对的时间复杂度为 $O(N^4)$ ，计算量达到 10^{12} ，会超时。我们需要使用 **树状数组（Binary Indexed Tree / Fenwick Tree）** 或 **归并排序** 将计算逆序对的时间复杂度降低到 $O(N^2 \log(N^2))$ 。

Python 代码实现

```

1 def merge_count(arr):
2     if len(arr) <= 1:
3         return arr, 0
4     mid = len(arr) // 2
5     left, a = merge_count(arr[:mid])
6     right, b = merge_count(arr[mid:])
7     merged = []
8     i = j = 0
9     inv = a + b
10    while i < len(left) and j < len(right):
11        if left[i] <= right[j]:
12            merged.append(left[i])
13            i += 1
14        else:
15            merged.append(right[j])
16            j += 1
17            inv += len(left) - i
18    merged += left[i:]
19    merged += right[j:]
20    return merged, inv
21
22 def main():
23     import sys
24     data = sys.stdin.read().split()
25     if not data:
26         return
27     n = int(data[0])
28     grid = []
29     index = 1
30     blank_row = -1
31     for i in range(n):
32         row = list(map(int, data[index:index+n]))
33         index += n
34         grid.append(row)
35         if 0 in row:
36             blank_row = i
37
38     flat = []
39     for row in grid:
40         for x in row:
41             if x != 0:
42                 flat.append(x)
43
44     _, inv = merge_count(flat)
45
46     if n % 2 == 1:
47         ans = "yes" if inv % 2 == 0 else "no"
48     else:
49         ans = "yes" if (inv + blank_row) % 2 == 1 else "no"
50     print(ans)
51
52 if __name__ == "__main__":
53     main()

```

```

1  import sys
2
3  # 定义树状数组类，用于快速计算逆序对
4  class FenwickTree:
5      def __init__(self, size):
6          self.tree = [0] * (size + 1)
7
8      def update(self, i, delta):
9          while i < len(self.tree):
10             self.tree[i] += delta
11             i += i & (-i)
12
13      def query(self, i):
14          s = 0
15          while i > 0:
16             s += self.tree[i]
17             i -= i & (-i)
18          return s
19
20  def solve():
21      # 使用 sys.stdin.read 读取所有输入，处理多行和空格
22      input_data = sys.stdin.read().split()
23
24      if not input_data:
25          return
26
27      iterator = iter(input_data)
28      try:
29          n = int(next(iterator))
30      except StopIteration:
31          return
32
33      flat_list = []
34      zero_row = -1
35
36      # 读取盘面数据
37      # 同时记录 0 所在的行，并构建去除 0 后的扁平数组
38      for r in range(n):
39          for c in range(n):
40              val = int(next(iterator))
41              if val == 0:
42                  zero_row = r # 记录 0 所在的行号 (0-indexed)
43              else:
44                  flat_list.append(val)
45
46      # 计算逆序对
47      # 数字范围是 1 到 n*n-1
48      max_val = n * n
49      bit = FenwickTree(max_val)
50      inversions = 0
51
52      for i, x in enumerate(flat_list):
53          # i 是当前已经遍历过的数字个数
54          # bit.query(x) 是当前遍历过的数字中 <= x 的个数
55          # i - bit.query(x) 就是当前遍历过的数字中 > x 的个数 (即逆序对)

```

```

56     inversions += (i - bit.query(x))
57     bit.update(x, 1)
58
59     # 根据 N 的奇偶性进行判定
60     if n % 2 == 1:
61         # N 是奇数: 逆序对必须是偶数
62         if inversions % 2 == 0:
63             print("yes")
64         else:
65             print("no")
66     else:
67         # N 是偶数: (逆序对 + 0距离底部的行数) 必须是偶数
68         dist_from_bottom = (n - 1) - zero_row
69         if (inversions + dist_from_bottom) % 2 == 0:
70             print("yes")
71         else:
72             print("no")
73
74 if __name__ == "__main__":
75     solve()

```

代码解析

1. 输入处理：使用 `sys.stdin.read().split()` 一次性读取所有数据，这比 `input()` 快且能自动处理换行符和空格。
2. 树状数组 (BIT):
 - `update(x, 1)`: 将数字 `x` 出现次数加 1。
 - `query(x)`: 查询小于等于 `x` 的数字出现过多少次。
 - `i - query(x)`: 利用前缀和思想，计算出比当前数字 `x` 大的已出现数字个数，这正是逆序对的定义。
3. 逻辑判定:
 - 奇数阶：左右移动不改变逆序对，上下移动空格跨越 $N - 1$ (偶数) 个数字，逆序对奇偶性不变。目标状态逆序对为 0 (偶数)，所以当前逆序对必须为偶数。
 - 偶数阶：上下移动空格跨越 $N - 1$ (奇数) 个数字，逆序对奇偶性改变。每向下移动一行，逆序对奇偶性反转一次。我们需要保证：(当前逆序对 + 将空格移到底部所需的步数) 的奇偶性 == 目标状态(0)的奇偶性。

练习T数字华容道 (Hard Version)

Cycle Decomposition, <http://cs101.openjudge.cn/practice/30179/>

总时间限制：1000ms，单个样例点时间限制：500ms，内存限制：65536kB

数字华容道的游戏玩法：在一个 $n \times n$ 的方格中放置 1 到 $n^2 - 1$ 的所有数字（每个数字占据一空间），剩下一个空格（输入中用 0 表示）。每次可以将与空格相邻的几个数字平移到空格处。游戏目标是尽可能快的将给定盘面还原到初始状态。（初始状态就是 1 到 $n^2 - 1$ 按顺序填充每一行，空格在右下角的盘面）

例如一个可能的初始盘面是：

```
1 | 1 2 3 4
2 | 5 6 7 8
3 | 9 10 11 12
4 | 15 13 14 0
```

你的目标是通过平移数字，将盘面变成

```
1 | 1 2 3 4
2 | 5 6 7 8
3 | 9 10 11 12
4 | 13 14 15 0
```

的样子。

一天，小K正准备玩数字华容道，但是他不小心把盘面中的数字小方格都打翻在地了。虽然他将所有的数字都乱序排入了盘面中，但是这样的盘面是不一定有解的，所以他想请你判断一下这个盘面是否有解。

输入

第 1 行：一个整数 t ，($1 \leq t \leq 100$)，表示样例数目。

接下来对于每个样例：

第 1 行：一个整数 n ，($2 \leq n \leq 1000$)。（你就不要问小K是怎么把一百万个小方格捡起来的了）

第 2 行到第 $n + 1$ 行： n 个整数，表示盘面。

数据保证每一个测试点都有 $\max(n^2) \times t \leq 10^7$ 。

输出

输出 t 行。对于每一个样例，如果盘面有解，输出yes，否则输出no

样例输入

```
1 | 2
2 | 4
3 | 1 2 3 4
4 | 5 6 7 8
5 | 9 10 11 12
6 | 13 14 15 0
7 | 2
8 | 2 1
9 | 3 0
```

样例输出

```
1 | yes
2 | no
```

(注：此题与Easy Version唯一的不同之处在于Hard Version的每一个输入文件都有多个样例，而Easy Version只有一个。)

提示：

tags: math, cycle decomposition

hint: 一个经典的智力问题。

现在有100个囚犯被囚禁，典狱长准备和他们玩一个游戏：有一百张写有 1 — 100 号码的纸条被倒扣在一百个编号 1 — 100 的盒子中。每个囚犯可以打开 50 个盒子，但是所有囚犯在打开完盒子后不能与其他囚犯交流。如果所有囚犯都能找到自己的号码对应的纸条，他们就能成功逃脱。否则他们就都会被囚禁终生。如果每个囚犯都只是随机的打开 50 个盒子，则他们能逃生的概率仅为 0.5^{100} ，微乎其微。但是一个聪明的囚犯提出了一个方案：每个人都打开自己的号码对应的盒子，找到下面纸条上的号码，然后打开对应号码的盒子，如此往复，直到找到自己的号码为止。这样做可以显著的将逃生概率增加到 30% 左右。

数学上，我们可以把从一个数移动到这个数对应位置的下一个数，如此往复，最终连会自己的结构称为循环 (cycle)。比如说 2 3 1 就是一个循环，它实际表示了 $2 \rightarrow 3 \rightarrow 1 \rightarrow 2$ 在囚犯问题中，所有号码相当于组成了一个全排列。所有囚犯寻找自己号码的过程就相当于在全排列的各个不同循环中移动一圈。可以证明，一个全排列可以被唯一分解为许多不相交（不共用元素）的循环。

循环的性质：一个有 n 个数的循环可以通过 $n - 1$ 次元素交换变回递增的序列。比如循环 2 3 1 就可以通过先交换 2 和 3，再交换 3 和 1 得到递增序列 1 2 3。

针对数字华容道 (N -Puzzle) 问题，判定是否有解的标准结论如下：

1. 当 N 为奇数时：盘面有解 $\iff$ 逆序对数（或交换次数）为偶数。
2. 当 N 为偶数时：盘面有解 $\iff$ （逆序对数 + 空格所在行从下往上的行号）为奇数。
(注：这里的行号是从 1 开始计数的，目标状态空格在右下角，即第 1 行)

重点：

1. 内存管理： $N = 1000$ 时 $N^2 = 10^6$ 。使用 `array.array` 存储 10^6 个整数仅需 4MB，比 `list` 节省约 90% 内存。
2. 输入效率： 10^7 个数字的读取对 Python 来说极具挑战。使用 `itertools.islice` 配合 `map(int, ...)` 是最快的方式。

优化后的代码：

```
1  import sys
2  import array
3  import itertools
4
5  def solve():
6      # 1. 使用生成器处理输入，避免一次性加载导致内存超限
7      def get_tokens():
8          for line in sys.stdin:
9              for word in line.split():
10                 yield word
11
12     tokens = get_tokens()
13
14     # 获取测试用例数量 t
15     first_token = next(tokens, None)
16     if first_token is None:
17         return
18     t = int(first_token)
19
20     # 将标准库函数映射为局部变量，提高循环内的调用速度
21     write = sys.stdout.write
```

```

22
23     for _ in range(t):
24         n_str = next(tokens, None)
25         if n_str is None:
26             break
27         n = int(n_str)
28         total = n * n
29
30         # 2. 使用 array.array 节省内存 ('I' 表示无符号 4 字节整数)
31         # 预分配空间比动态 extend 更快
32         grid_full = array.array('I', map(int, itertools.islice(tokens,
total)))
33
34         # 3. 寻找 0 的位置并计算行号
35         zero_idx = -1
36         for i in range(total):
37             if grid_full[i] == 0:
38                 zero_idx = i
39                 break
40
41         # 空格所在行 (从下往上数, 最底下一行是第 1 行)
42         # zero_idx // n 是从上往下数的行索引 (0 ~ n-1)
43         row_from_bottom = n - (zero_idx // n)
44
45         # 4. 构造不含 0 的紧凑序列用于计算置换环
46         # 为了极速处理, 手动将 0 后的元素前移
47         grid = array.array('I', [0] * (total - 1))
48         for i in range(zero_idx):
49             grid[i] = grid_full[i]
50         for i in range(zero_idx + 1, total):
51             grid[i-1] = grid_full[i]
52
53         # 释放大数组内存
54         del grid_full
55
56         # 5. 置换环分解计算交换次数的奇偶性
57         # 时间复杂度  $O(N^2)$ , 对 Python 1s 限制来说  $10^7$  次操作是极限
58         num_elements = total - 1
59         visited = bytearray(num_elements)
60         cycles = 0
61
62         # 局部变量优化
63         g = grid
64         v = visited
65
66         for i in range(num_elements):
67             if not v[i]:
68                 cycles += 1
69                 curr = i
70                 while not v[curr]:
71                     v[curr] = 1
72                     # 目标状态值 v 在索引 v-1 处
73                     curr = g[curr] - 1
74
75         # 交换次数 parity = (总数 - 环数) % 2

```



```

76     swaps_even = (num_elements - cycles) % 2 == 0
77
78     # 6. 核心判定逻辑
79     if n % 2 == 1:
80         # N为奇数：交换次数必须为偶数
81         write("yes\n" if swaps_even else "no\n")
82     else:
83         # N为偶数：(交换次数 + 空格行号) 必须为奇数
84         # 逻辑等价于：swaps_even 的布尔值必须与 (row_from_bottom 为奇数) 的布
    尔值一致
85         row_odd = (row_from_bottom % 2 == 1)
86         if swaps_even == row_odd:
87             write("yes\n")
88         else:
89             write("no\n")
90
91     # 显式清理
92     del grid
93     del v
94
95 if __name__ == "__main__":
96     solve()

```

关键点说明：

- **内存 (Memory)：** 通过 `array.array` 和 `bytearray`，即使在 $N = 1000$ 的情况下，核心数据结构占用的内存仅约 5MB。加上 Python 解释器本身的开销，总内存会稳定在 30MB-40MB 左右，远低于 64MB 的限制。
- **时间 (Time)：**
 - 使用 `itertools.islice` 和 `map` 绕过了 Python 慢速的显式 `for` 循环读取。
 - 置换环分解通过 `bytearray` 进行标记，时间复杂度为 $O(N^2)$ 。由于题目保证总元素 $\sum N^2 \leq 10^7$ ，在 Python 中通过局部变量优化后，可以在 1 秒内完成。
- **偶数阶判定逻辑：**
 - 目标状态的空格行号是 1（奇数），逆序对是 0（偶数），和为奇数。
 - 因此，任何状态只要（交换次数 + 从下往上行号）的和为奇数，即为有解。
 - 代码中的 `if swaps_even == row_odd` 完美涵盖了：
 - 偶数 + 奇数 = 奇数 (`True == True`)
 - 奇数 + 偶数 = 奇数 (`False == False`)

B. KMP, Manacher: Symmetry Power

Q: 我一直感觉KMP与Manacher算法很像，都是利用预处理好的信息，提高效率。Manacher算法更难理解，因为它的预处理也是online动态进行的，KMP的预处理只要一次。

KMP 算法和 Manacher 算法在底层哲学上确实是“同门师兄弟”。它们的核心思想都是：利用已知信息（对称性）来避免重复的暴力匹配，从而将时间复杂度从 $O(N^2)$ 降低到 $O(N)$ 。

觉得 Manacher 更难理解，是因为 Manacher 把“预处理”和“主循环”完全融合在了一起，产生了一种“边走边算、边算边用”的动态感。

可以从以下几个维度来深度对比这两个算法，来理清思路：

1. 核心哲学的对比：利用什么样的“对称性”？

- **KMP 算法：利用“前后缀”的对称性。**
 - 当你匹配失败时，KMP 告诉你：虽然现在没对上，但由于模式串的前缀和当前已匹配部分的后缀是**完全一样的**，你不需要从头开始，直接跳到前缀位置继续即可。
 - **核心：**节省的是“模式串回溯”的时间。
- **Manacher 算法：利用“镜像中心”的对称性。**
 - 当你计算某个点的回文半径时，Manacher 告诉你：由于这个点处于一个更大的回文串内部，它的情况一定和它关于中心对称的那个“兄弟点”**高度相似**。
 - **核心：**节省的是“回文半径从 1 开始增长”的时间。

2. 关于你提到的“预处理”差异

你说 KMP 的预处理是一次性的，而 Manacher 是动态的。这其实是一个很有意思的视角，我们可以这样拆解：

KMP 的“伪”分离

KMP 计算 `next` 数组的过程，其实本质上是**模式串自己匹配自己**。

- 如果你把 `next` 数组的构建看作是一个独立的过程，它确实是“预处理”。
- 但如果你看代码逻辑，计算 `next[i]` 时利用了 `next[0...i-1]` 的信息，这和 Manacher 计算 `p[i]` 时利用 `p[0...i-1]` 的信息在数学本质上是一样的。

Manacher 的“动态性”

Manacher 维护了两个关键变量：`C`（当前最右回文边界的中心）和 `R`（当前最右回文边界的右端点）。

- 它的难点在于：随着 `i` 的往后移动，`R` 也在不断地被更新。
- 这种“Online”的感觉来源于：每一时刻的计算都极度依赖于此时此刻 `R` 覆盖到了哪里。如果 `i` 在 `R` 内部，我们就“白嫖”镜像点的信息；如果 `i` 走出了 `R`，我们就不得不通过暴力扩展来更新 `R`。

3. 为什么它们都能达到 $O(N)$ ？（线性效率的真相）

虽然这两个算法逻辑不同，但它们时间复杂度证明的思路是一模一样的，都叫“摊还分析”：

- **KMP：**主串的指针 `i` 从不回退，模式串指针 `j` 虽然会跳，但 `j` 增加的总次数不会超过 `i` 增加的总次数。
- **Manacher：**关键在于 `R`（最右边界）。算法只有在 `i` 超过 `R` 或者当前回文尝试突破 `R` 时才会进行真正的字符比较。**每比较成功一次，`R` 就会至少向右移动一个单位。**因为 `R` 最多移动 N 次，所以总的比较次数也是 $O(N)$ 。

4. 形象类比：抄作业

- **KMP 像是“经验丰富的翻译员”：**

翻译错了没关系，我记得之前有一段翻译过，逻辑是一样的，我直接从那句接着翻，不用重头看。

- **Manacher 像是“照镜子的画师”：**

我要画右边的脸，既然左边的脸已经画好了，而且我知道整张脸是对称的，我就直接把左边的线条“拷贝”过来。只有当画到左边没画过的地方（超出边界 R ）时，我才需要抬头看一眼实物，画几笔新的。

总结：为什么 Manacher 让你觉得更难？

Manacher 确实比 KMP 复杂一点，原因有三：

1. **数据改造：** 为了处理奇偶长度，必须先插入 $\#$ 占位符。
2. **多重分类讨论：** 计算 $p[i]$ 时，要考虑 i 是否在 R 内，以及镜像点的回文半径是否超出了 $R - i$ 的范围。
3. **状态更新：** C 和 R 是一直在变动的，这种动态感增加了大脑的负荷。

建议：

理解 Manacher 时，不要去死记那几个 `if-else`，而是盯住那个 R （最右边界）。

所有的逻辑只有一句话：“只要我在 R 里面，我就能利用对称性少干活；一旦我要突破 R 了，我就必须老老实实去匹配，并顺手把 R 往右推。”

如果你能接受这一点，你会发现 Manacher 实际上就是一种带缓存机制的暴力搜索，这和 KMP 的本质是高度统一的。

会了 Manacher，看 KMP 就像看减配版

之所以觉得 Manacher 更难、更“动态”，是因为它在 KMP 的基础上增加了一个维度，并把“信息复用”玩到了极致。

可以从三个层面拆解你的感受，彻底把这两个算法串联起来：

1. 为什么说 Manacher 是 KMP 的“威力加强版”？

- **KMP 的核心：** 知道“一段后缀”等于“一段前缀”，所以匹配失败时，可以“瞬移”（利用 `next` 数组）。
- **Manacher 的核心：** 知道“右边这部分”等于“左边部分的镜像”，所以在算右边时，可以“直取”（利用已算好的 $p[j]$ ）。

难度差异的根源：

- **KMP 的跳跃是一维的：** 它只处理“从左往右”的平移对称性。
- **Manacher 的跳跃是二维/对称的：** 它处理的是“关于中心”的镜像对称性。为了处理这个镜像，Manacher 引入了 R （最右边界）和 C （中心点）两个变量，这比 KMP 仅需维护一个指针要复杂得多。

2. 关于“Online（在线）”与“预处理”的深度误区

你提到的“KMP 预处理只要一次”其实是一个观察视角的问题。

如果我们对比“KMP 构建 `next` 数组”和“Manacher 计算 `p` 数组”，你会发现它们惊人地相似：

- **KMP 构建 `next`**：本质是“字符串自己匹配自己”。它在算 `next[i]` 时，也在动态利用之前算好的 `next[0...i-1]`。
- **Manacher 计算 `p`**：也是“字符串自己匹配自己（的镜像）”。它在算 `p[i]` 时，也在动态利用之前算好的 `p[mirror]`。

为什么你会觉得 Manacher 难？

因为 KMP 的主逻辑（匹配过程）和预处理（构建 `next`）在很多教材里被分成了两块。而 Manacher 的计算过程就是它的预处理过程。

在 Manacher 里，你找不到一个独立的“匹配阶段”，它一边往右走，一边通过 `R` 的移动来探索未知领域，同时又在 `R` 的保护范围内疯狂“白嫖”镜像点的信息。

3. 理解了 Manacher，KMP 哪儿变简单了？

如果你能跨越 Manacher 的三座大山，KMP 确实会显得非常“简陋”：

1. “不走回头路”的本质：

Manacher 让你深刻理解了什么是摊还分析 (Amortized Analysis)。在 Manacher 里，真正耗时的“字符比较”只会发生在 `i > R` 的时候，且每比较成功一次，`R` 就会向右推一步。因为 `R` 最多走 N 步，所以总复杂度是 $O(N)$ 。

回到 KMP：你会瞬间秒懂为什么 `j = next[j]` 的 `while` 循环不会导致 $O(N^2)$ 。因为 `j` 增加的总次数是有限的，回退的总次数不可能超过增加的总次数。

2. “信息复用”的逻辑：

Manacher 教会了你：如果当前位置处于某个“已知合法区域”内，它的初始值就不应该是 0。

在 KMP 里，这就是 `next` 数组；在 Z-Algorithm（扩展 KMP）里，这就是 Z-box。

3. 边界处理的艺术：

Manacher 为了处理奇偶长度插入了 `#`，为了防止越界插入了 `^` 和 `$`。这种“哨兵思想”和“统一模型”的思想，是解决所有字符串问题的杀手锏。

总结

你觉得 Manacher 难，是因为它在动态维护一个“已知区域”的边界 `R`，而 KMP 的已知区域相对固定。

一句话总结它们的共同点：

KMP：“我已经对比过这部分了，我知道前面有一样的，别让我再比一遍。”

Manacher：“我已经对比过这部分了，我知道左边有镜像，别让我再比一遍。”

当你能把 Manacher 里的 `mirror` 映射和 KMP 里的 `next` 跳转看作同一种“利用对称性跳过重复劳动”的行为时，你的字符串算法水平就已经上了一个大台阶。

练习M5. 最长回文子串

DP, Center Expansion, Manacher, <https://leetcode.cn/problems/longest-palindromic-substring/>